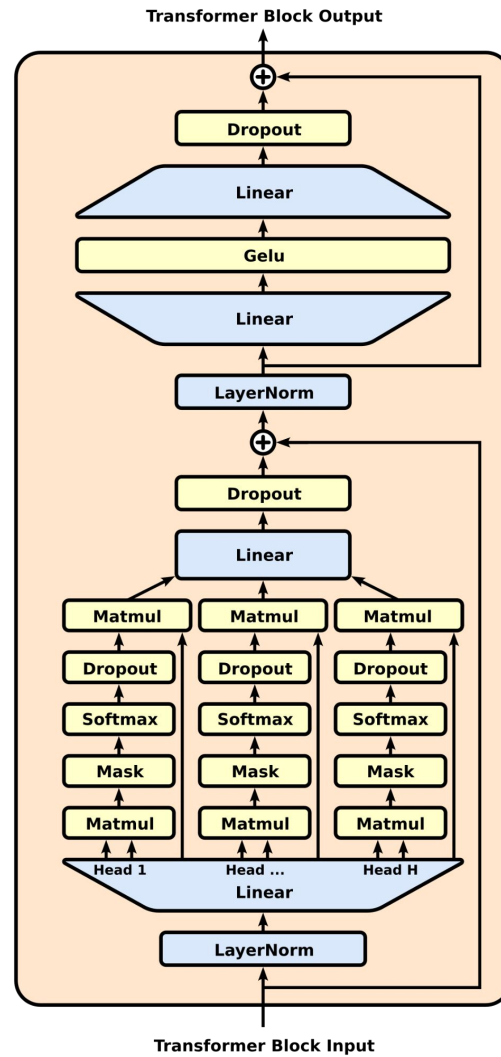
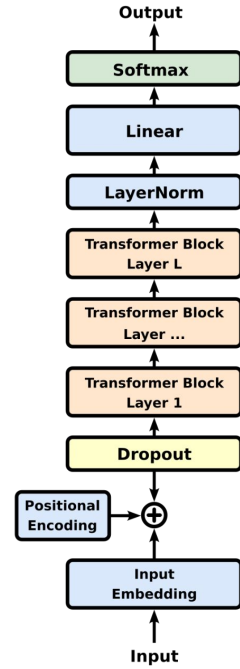


Оптимизация обучения БЯМ

План

1. Как ускорять подготовку данных
2. Как ускорять ллмки
 - a. Mixed precision
 - b. Kernels
3. Как впихнуть ллмку на видеокарту
4. Как понять, что ничего не поломалось



Full 4d parallelism

- Data
- Tensor(Sequence)
- Context
- Pipeline
- Expert

Memory requirements

Формула для подсчета параметров в ванильном трансформере:

$$N = h * v + L * (12 * h^2 + 13 * h) + 2 * h$$

Ну или вот так:

$$\begin{aligned} C &= \text{embed_layers} + \text{transformer_layers} + \text{other} \\ &= (\text{wte} + \text{wpe}) + L * (\text{ln_1} + \text{attn} + \text{ln_2} + \text{mlp}) + \text{ln_f} \\ &= (VE + PE) + L(2E + (4E^2 + 4E) + 2E + (2EH + E + H)) + (2E) \\ &= E(V + P) + L(12E^2 + 13E) + 2E \end{aligned}$$

Memory requirements

Сколько памяти нужно чтобы положить все необходимые для обучения параметры (в fp32)?

Memory requirements

Сколько памяти нужно чтобы положить все необходимые для обучения параметры (в fp32)?

$$m_{params} = 4 * N$$

$$m_{grad} = 4 * N$$

$$m_{opt} = (4 + 4) * N$$

Memory requirements

Сколько памяти нужно чтобы положить все необходимые для обучения параметры (в fp32)?

$$m_{params} = 4 * N$$

$$m_{grad} = 4 * N$$

$$m_{opt} = (4 + 4) * N$$

$$m_{params} = 2 * N$$

$$m_{grad} = 2 * N$$

$$m_{params_fp32} = 4 * N$$

$$m_{opt} = (4 + 4) * N$$

Memory requirements

Model parameters	FP32 or BF16 w/o FP32 grad acc	BF16 w/ FP32 grad acc
1B	16 GB	20 GB
7B	112 GB	140 GB
70B	1120 GB	1400 GB
405B	6480 GB	8100 GB

Memory requirements

А сколько памяти нужно для того, чтобы реально начать учиться?

Memory requirements

А сколько памяти нужно для того, чтобы реально начать учиться?

L - number of layers,

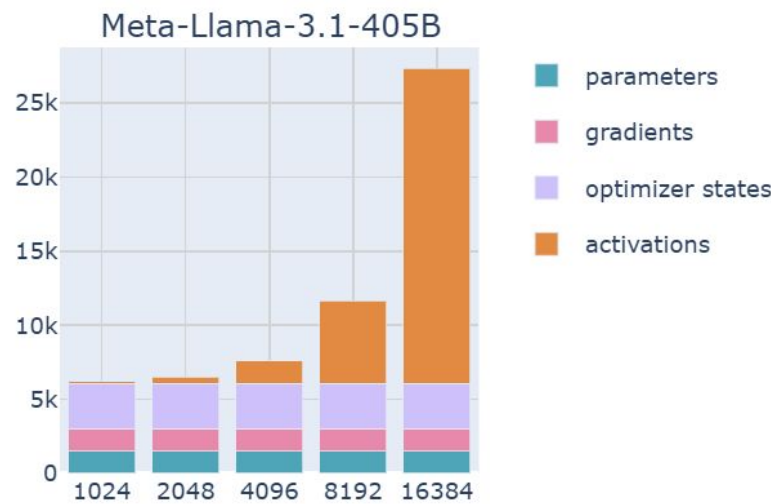
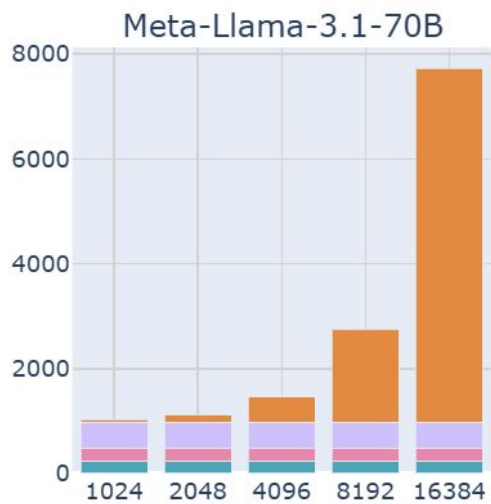
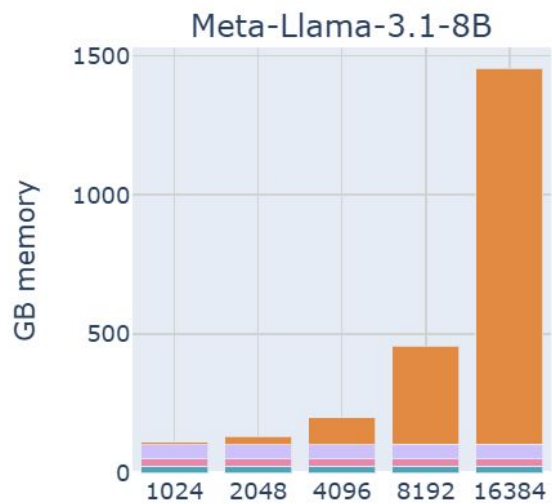
seq - sequence length,

bs - batch size in samples,

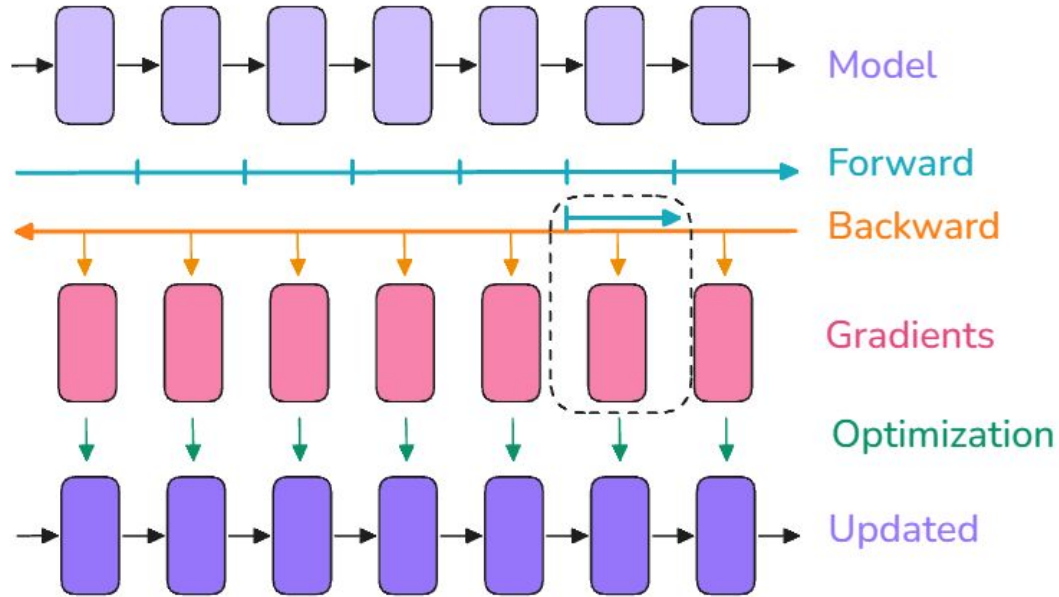
h - hidden dimension of the model

N_{head} - number of heads.

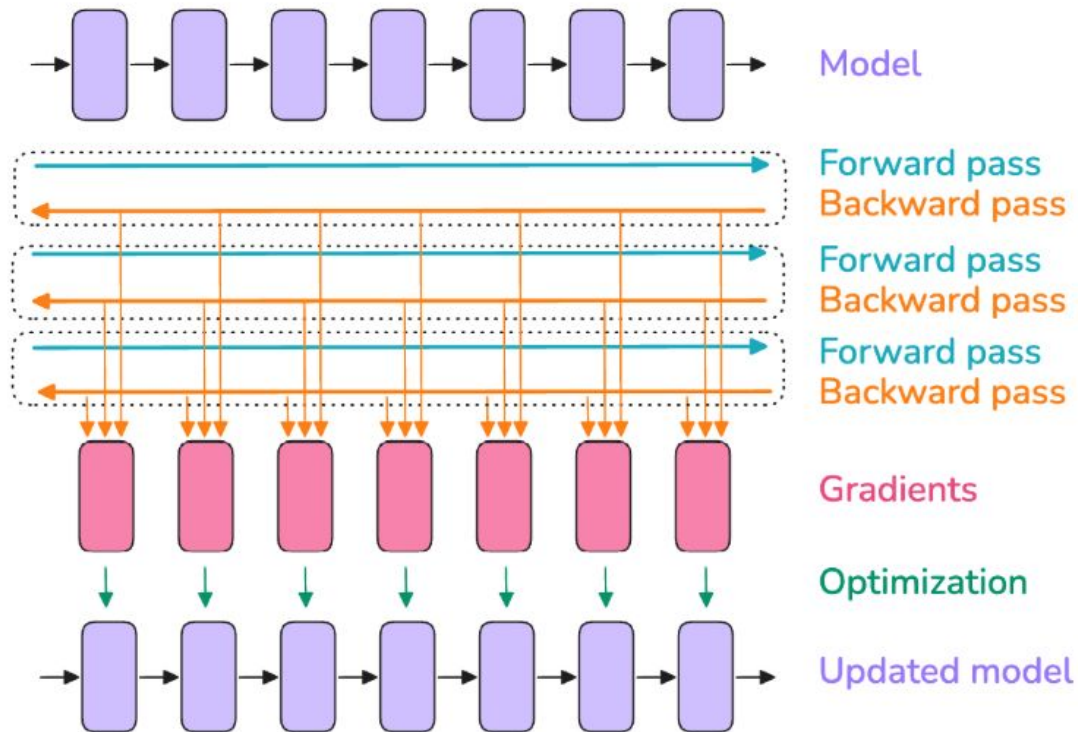
$$m_{act} = L \cdot seq \cdot bs \cdot h \cdot \left(34 + \frac{5 \cdot n_{heads} \cdot seq}{h} \right)$$

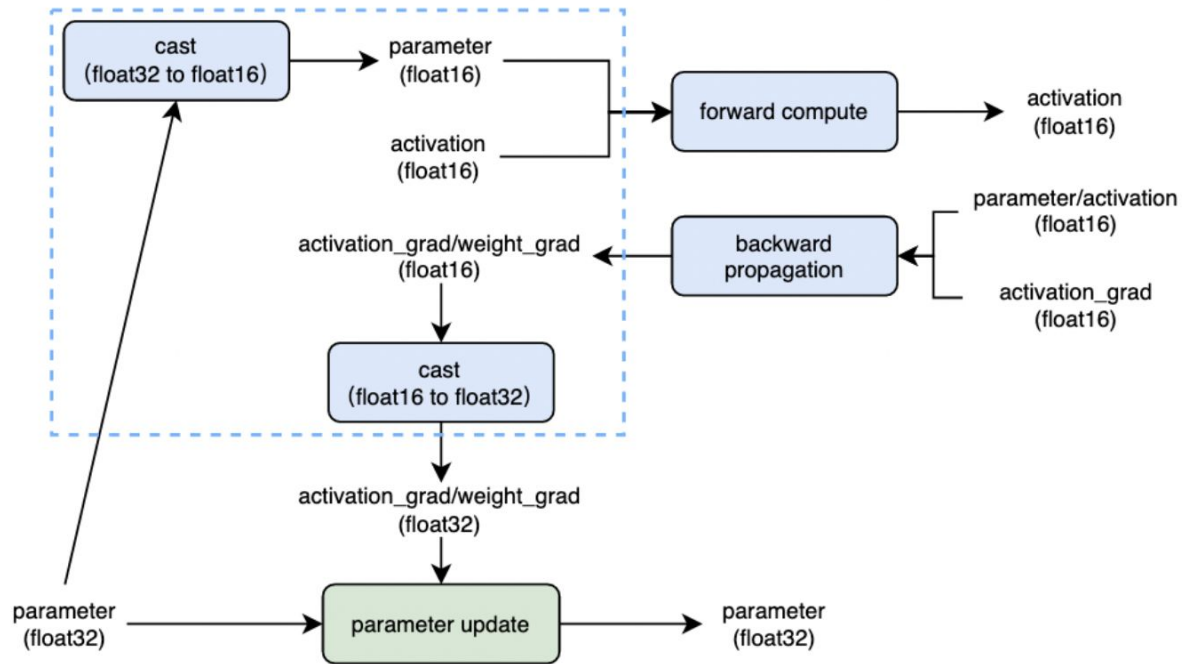


Activation recomputation / gradient checkpointing

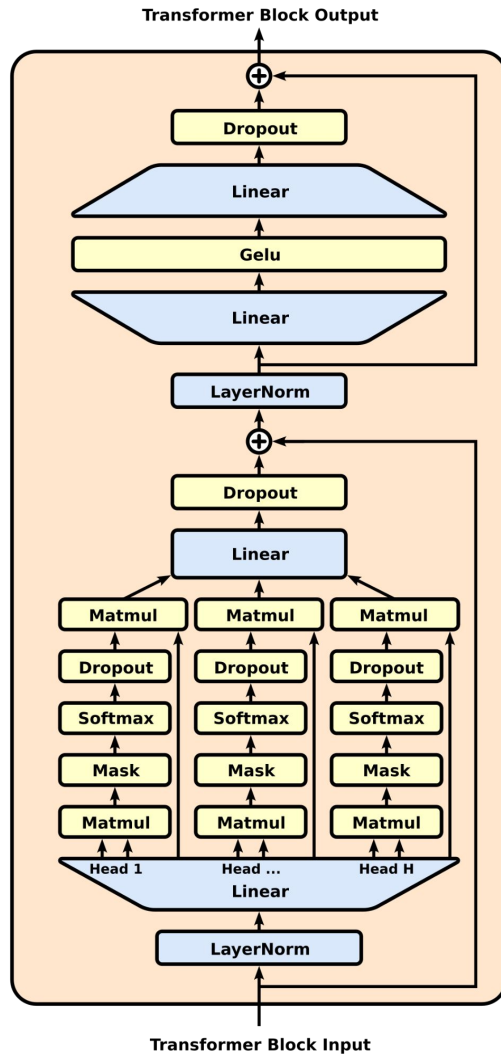
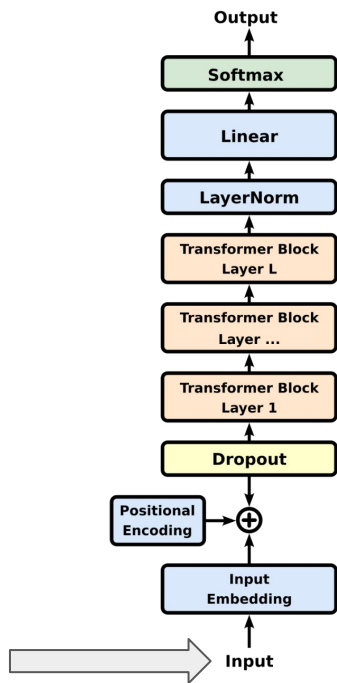


Gradient accumulation

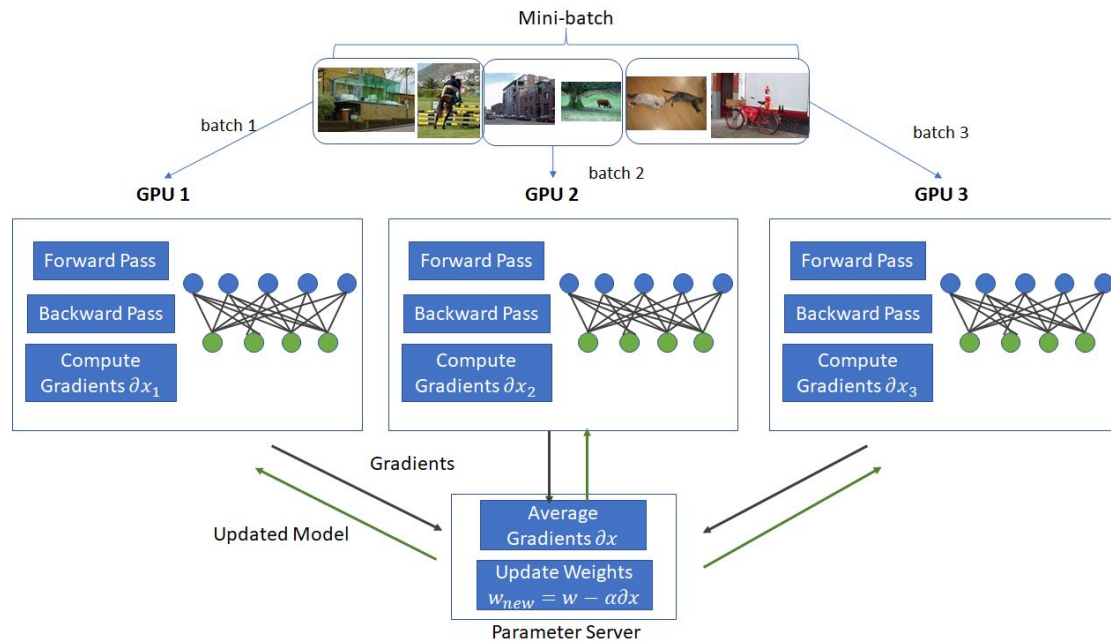




Data parallelism

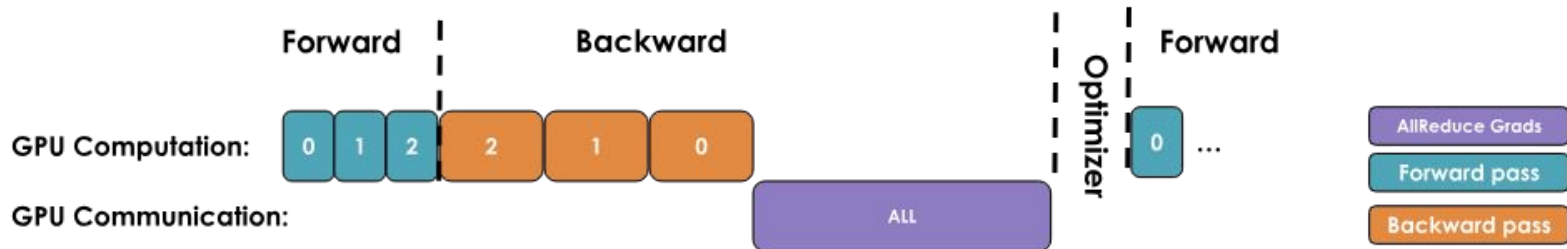


Data parallelism



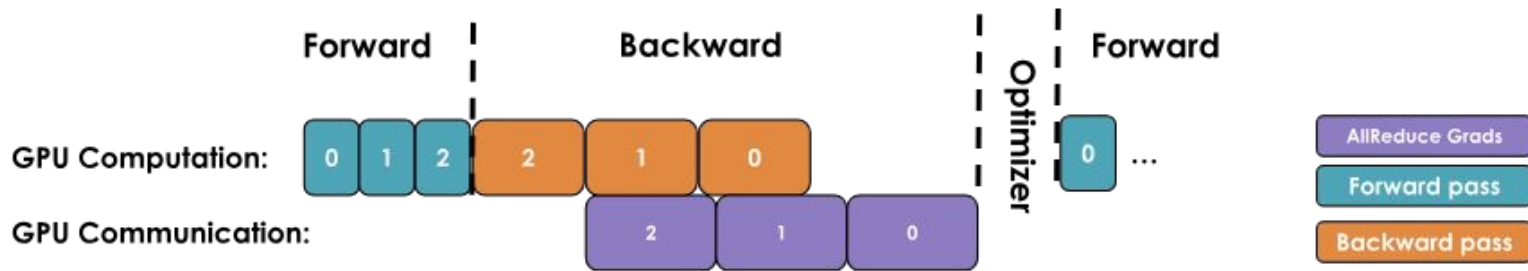
Data parallelism

Naive approach: all-reduce

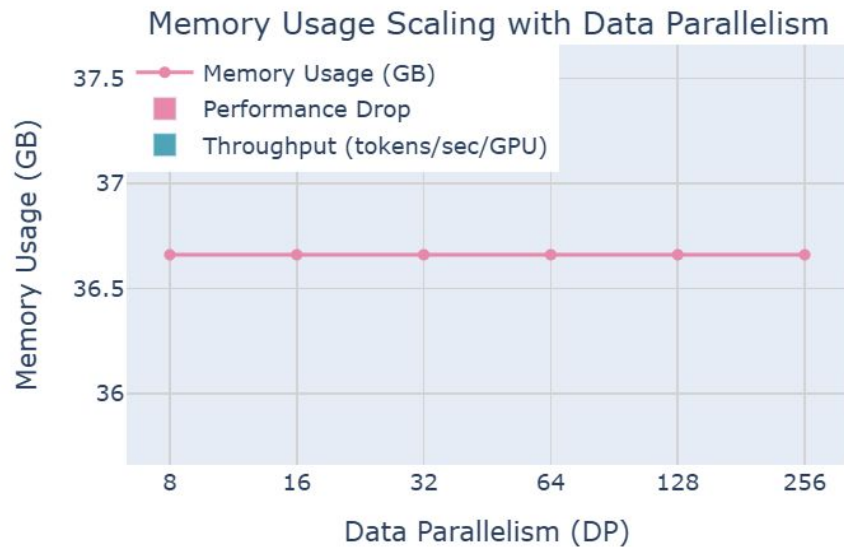
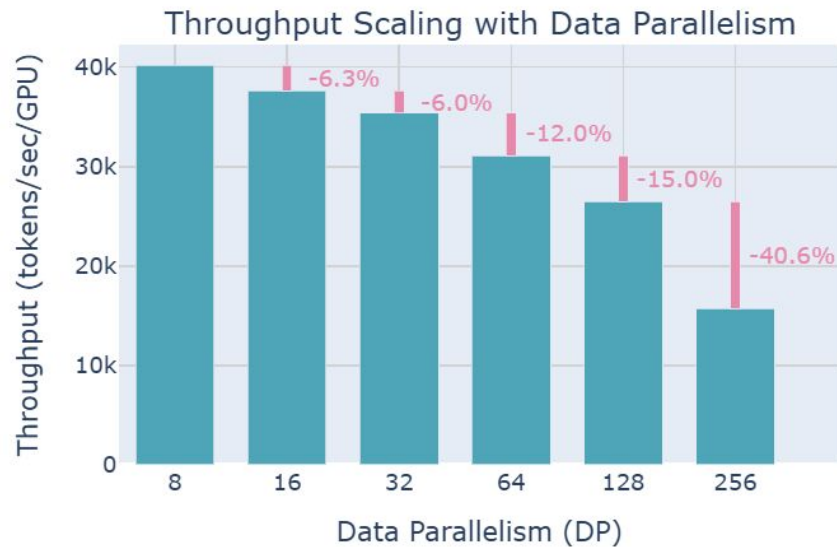


Data parallelism

Optimization - bucketing gradients:



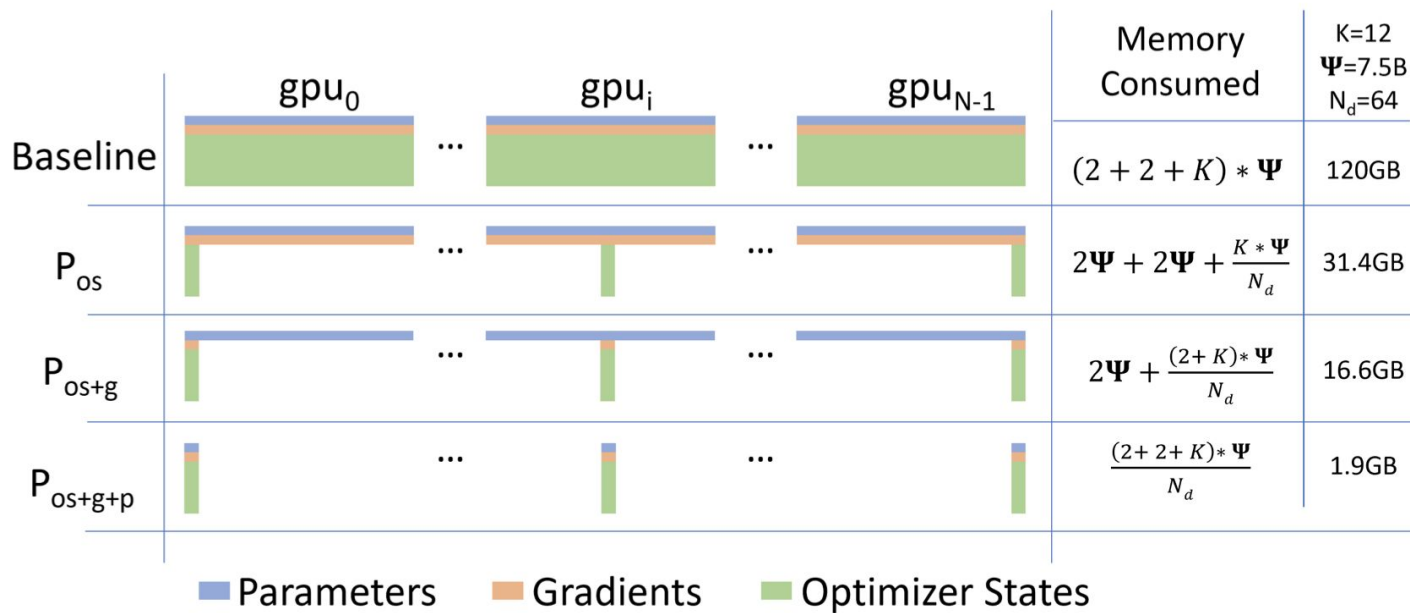
Data parallelism



Zero Redundancy Optimizer (ZeRO)

While data parallelism is an efficient way to scale training, the naive replication of optimizer states, gradients, and parameters across each DP rank introduces significant memory redundancy. ZeRO eliminates this by partitioning the optimizer states, gradients, and parameters across the data parallel dimension, while still allowing computation with the full set of parameters.

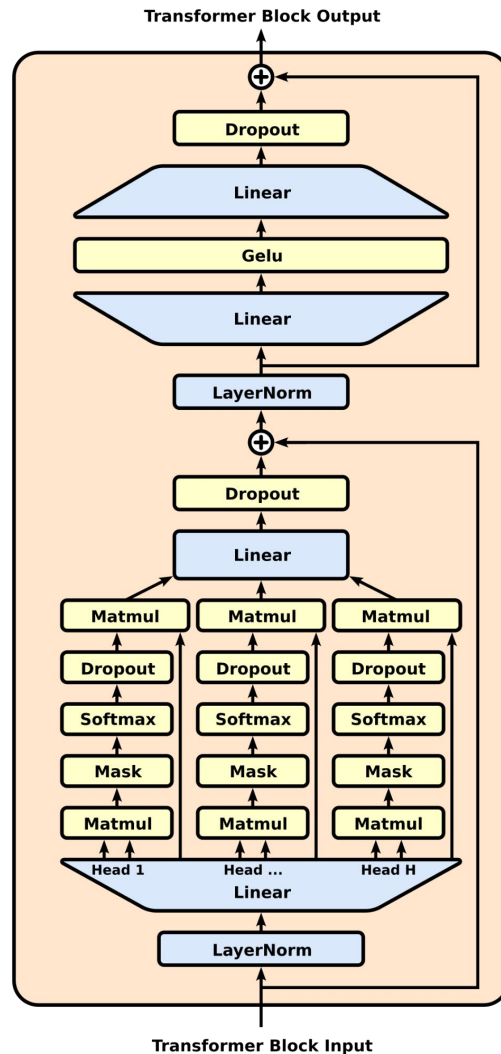
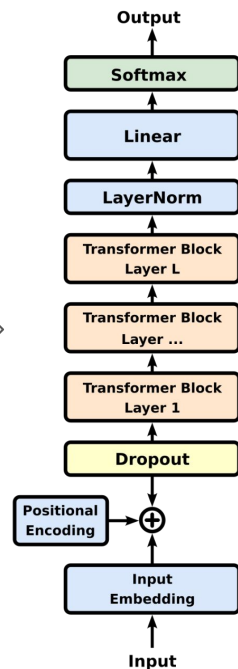
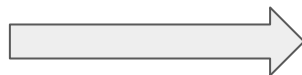
ZeRO



Pipeline parallelism

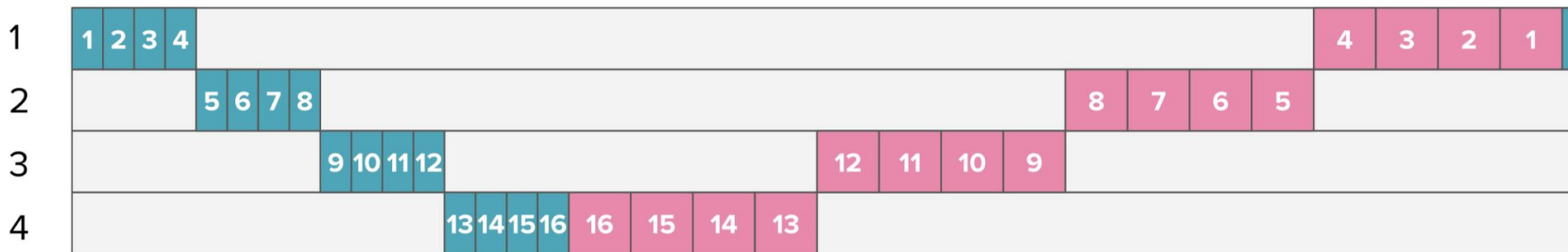
Pipeline parallelism is a simple but powerful technique - we split our model's layers across multiple GPUs! For example, if we have 8 GPUs, we could put layers 1-4 on GPU 1, layers 5-8 on GPU 2, and so on. This way, each GPU only needs to store and process a portion of the model's layers, significantly reducing the memory requirements per GPU.

Pipeline parallelism



Pipeline parallelism

GPU



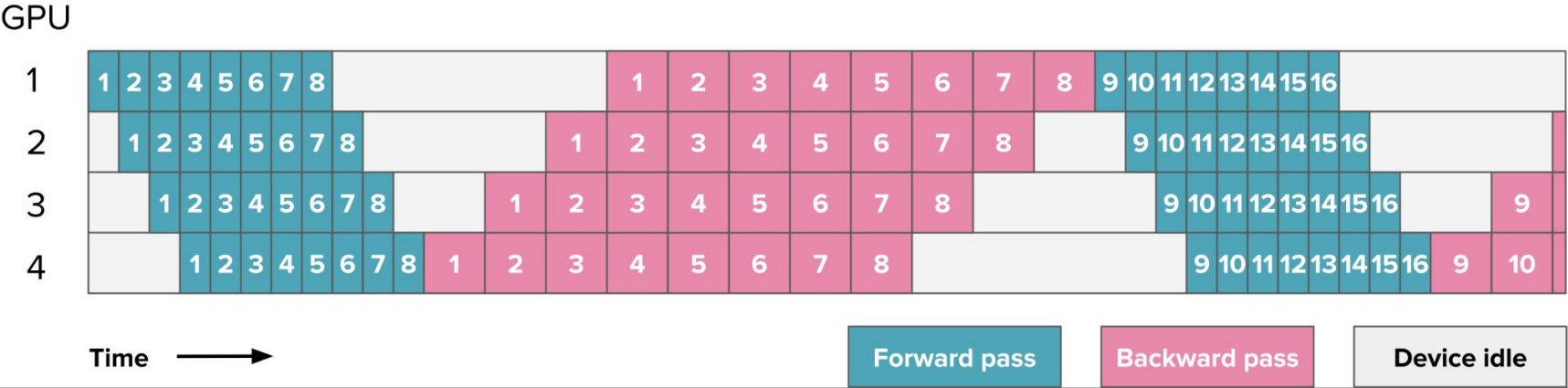
Time →

Forward pass

Backward pass

Device idle

Pipeline parallelism



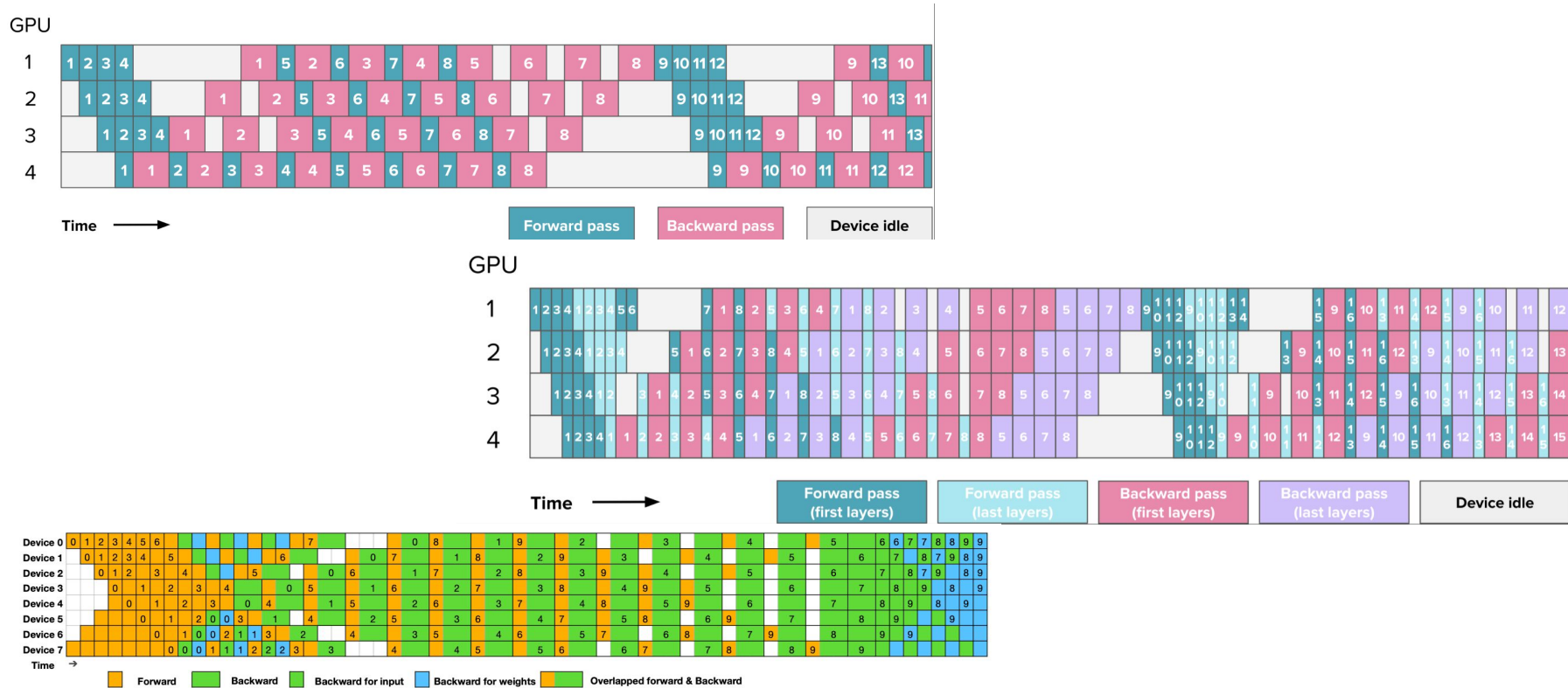
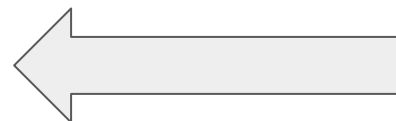
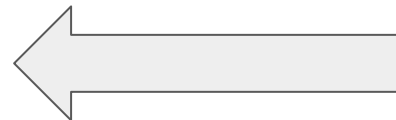
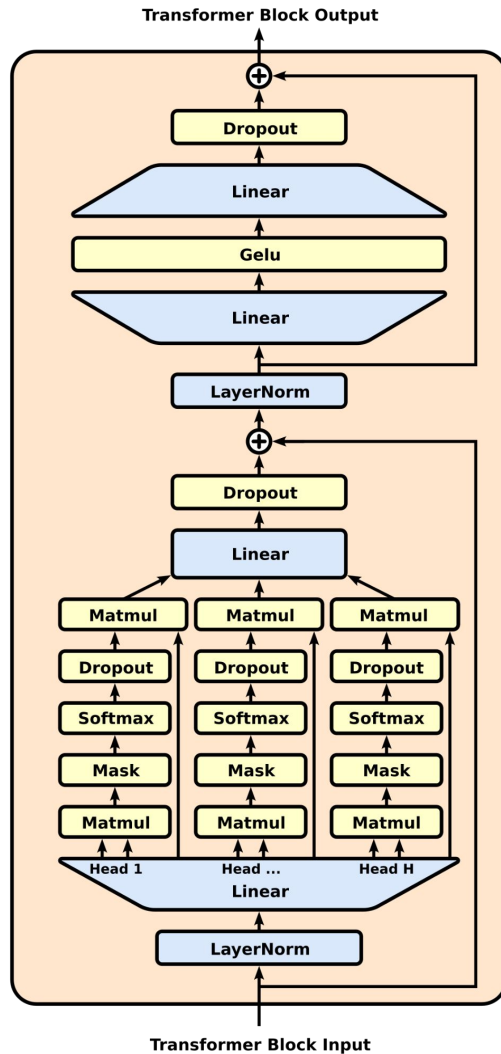
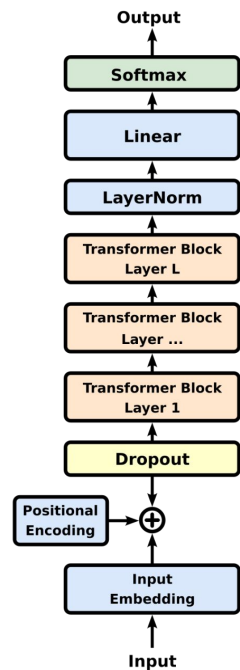


Figure 5 | Example DualPipe scheduling for 8 PP ranks and 20 micro-batches in two directions. The micro-batches in the reverse direction are symmetric to those in the forward direction, so we omit their batch ID for illustration simplicity. Two cells enclosed by a shared black border have mutually overlapped computation and communication.

Tensor parallelism



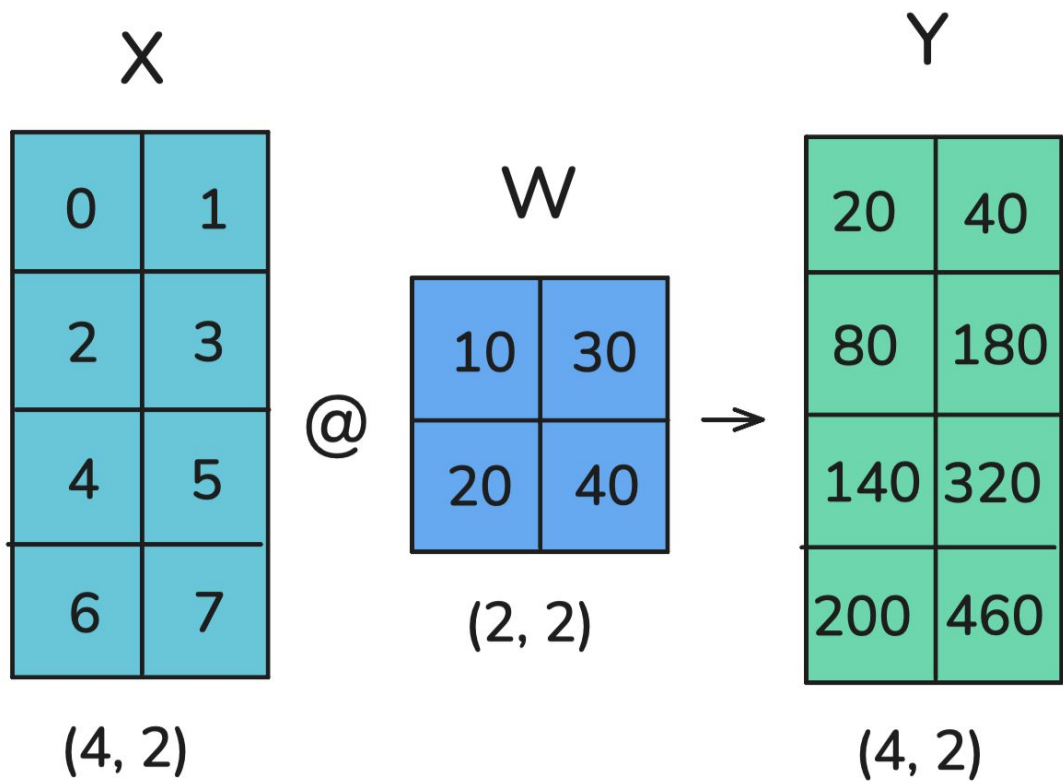
Tensor parallelism

Tensor parallelism leverages the mathematical properties of matrix multiplication, $A \times B$. To understand how it works, let's examine two fundamental equations that make this parallelization possible:

$$1. \quad A \cdot B = A \cdot [B_1 \quad B_2 \quad \dots] = [AB_1 \quad AB_2 \quad \dots]$$

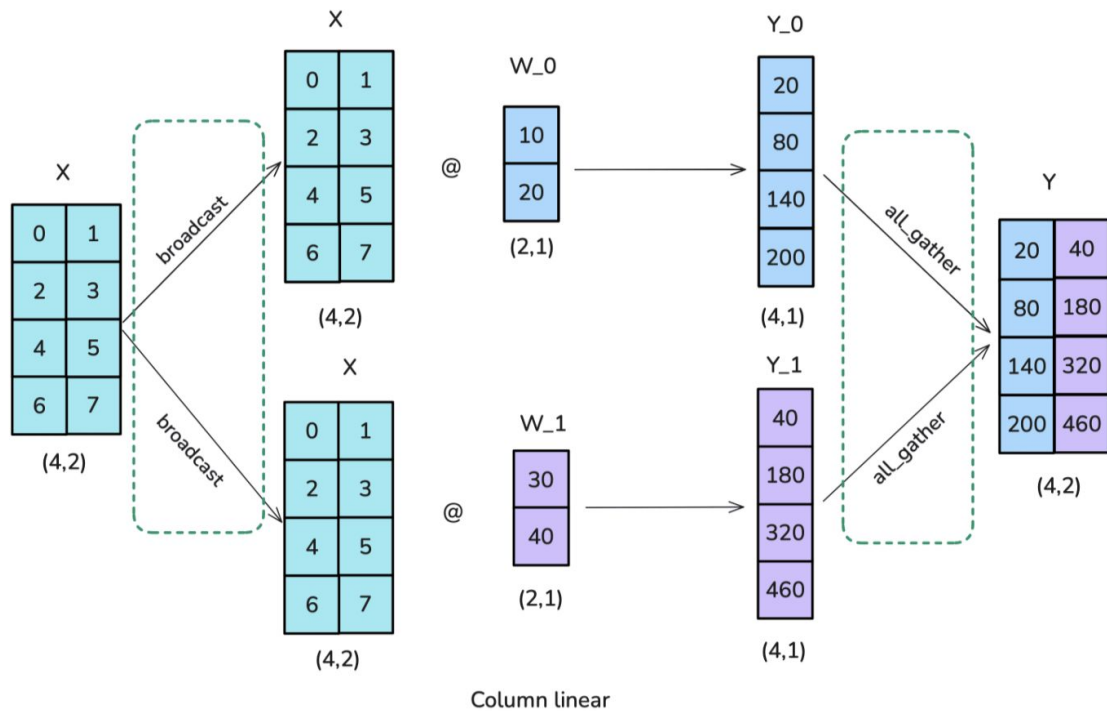
$$2. \quad A \cdot B = [A_1 \quad A_2 \quad \dots] \begin{bmatrix} B_1 \\ B_2 \\ \vdots \end{bmatrix} = \sum_{i=1}^n A_i B_i$$

Tensor parallelism



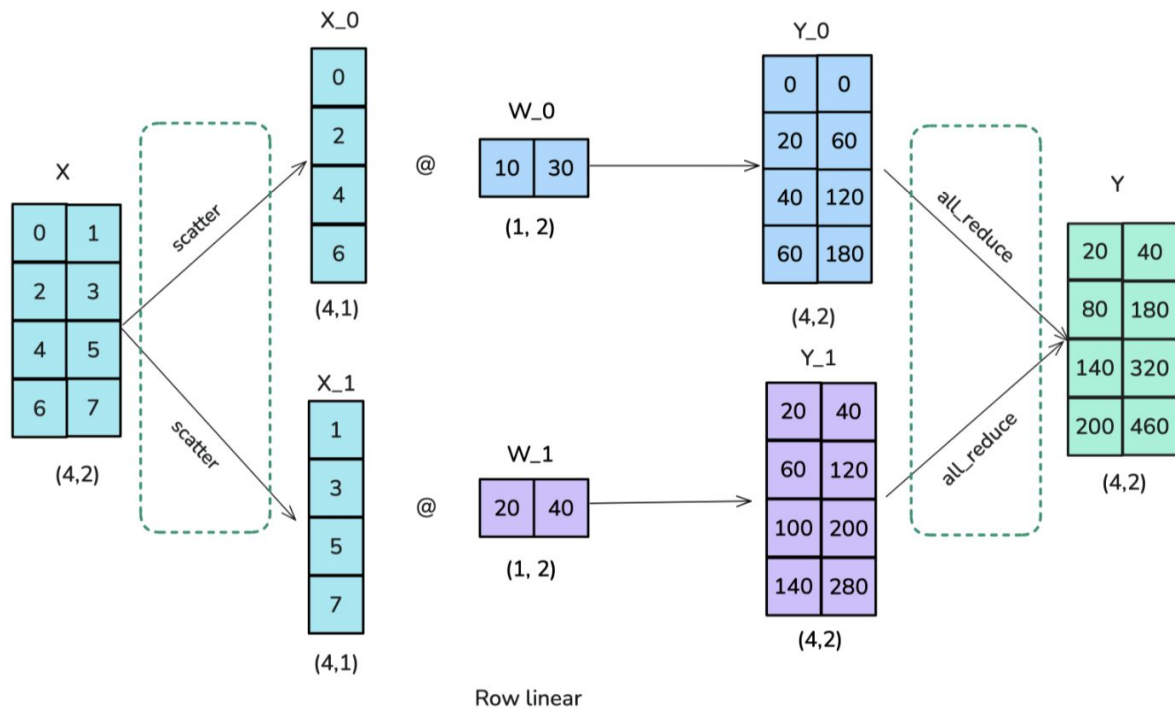
Tensor parallelism

Column-wise

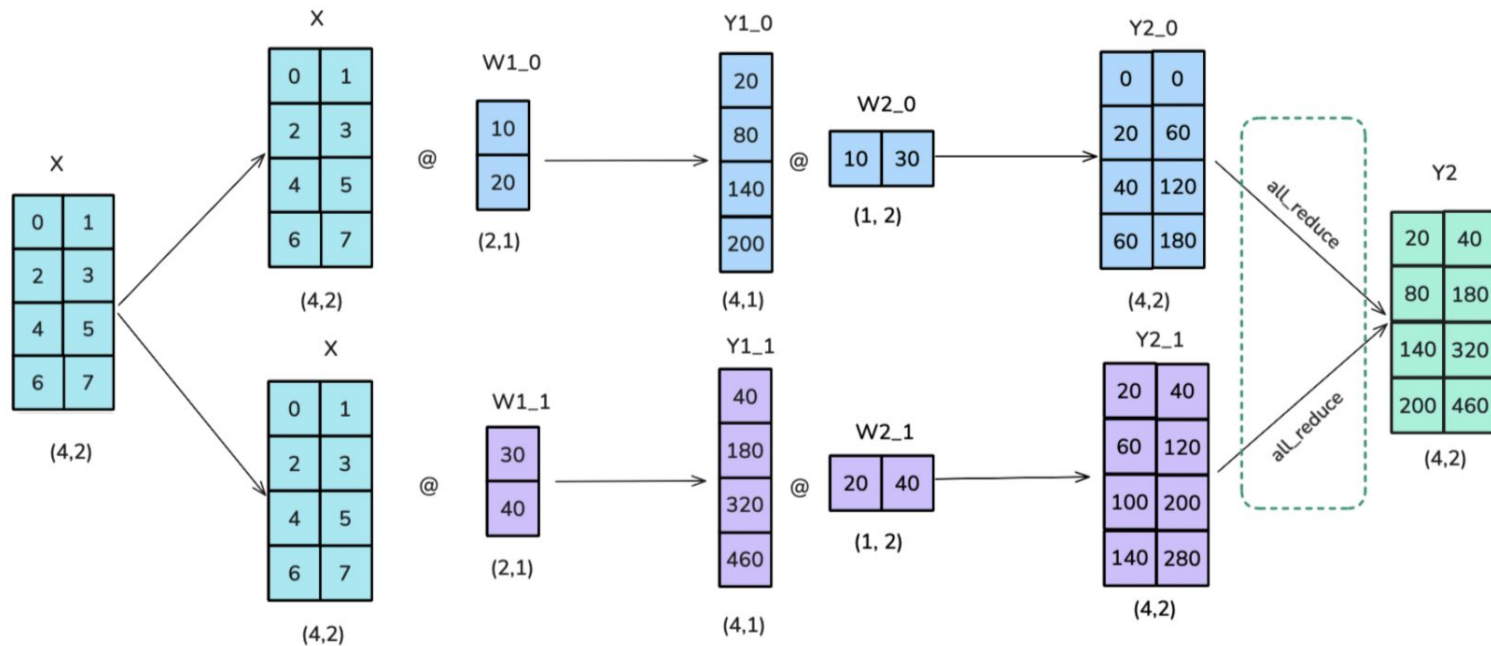


Tensor parallelism

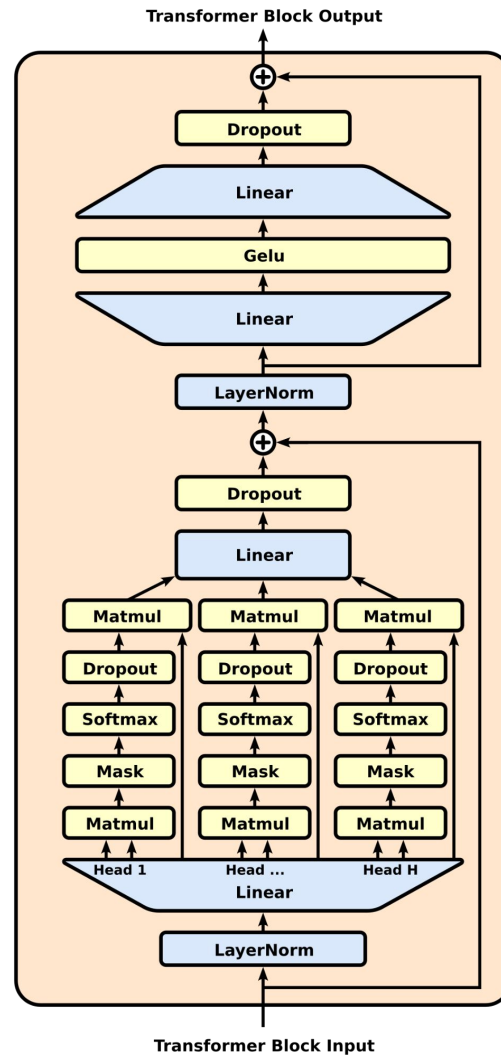
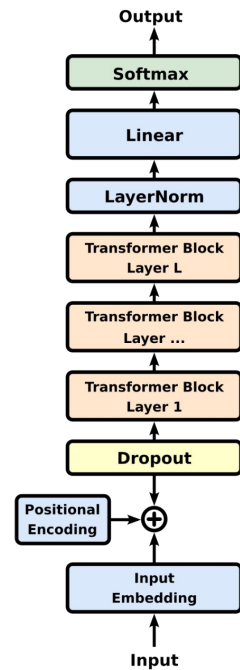
row-wise



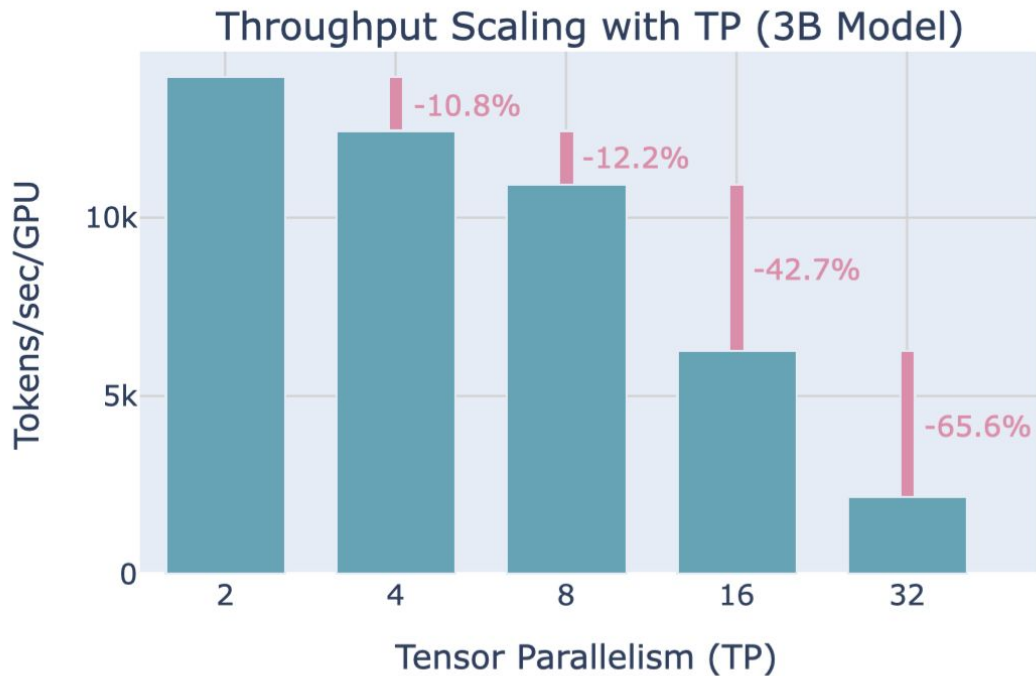
Tensor parallelism



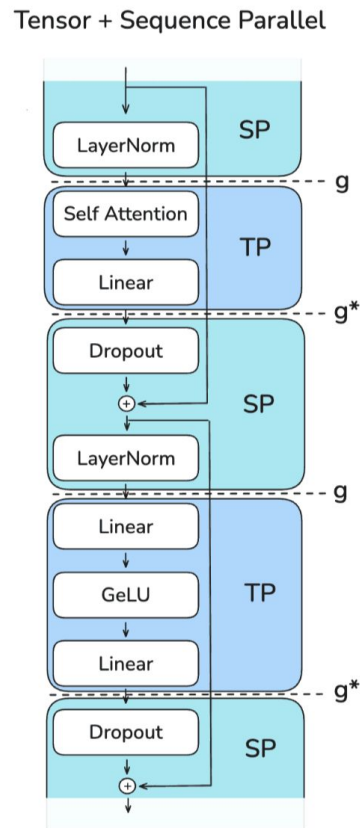
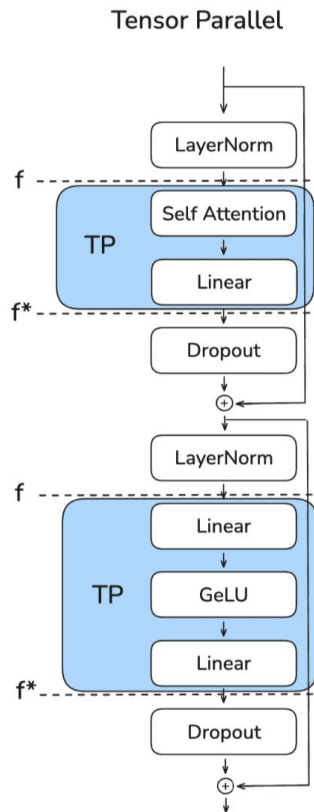
Tensor parallelism with column linear + row Linear



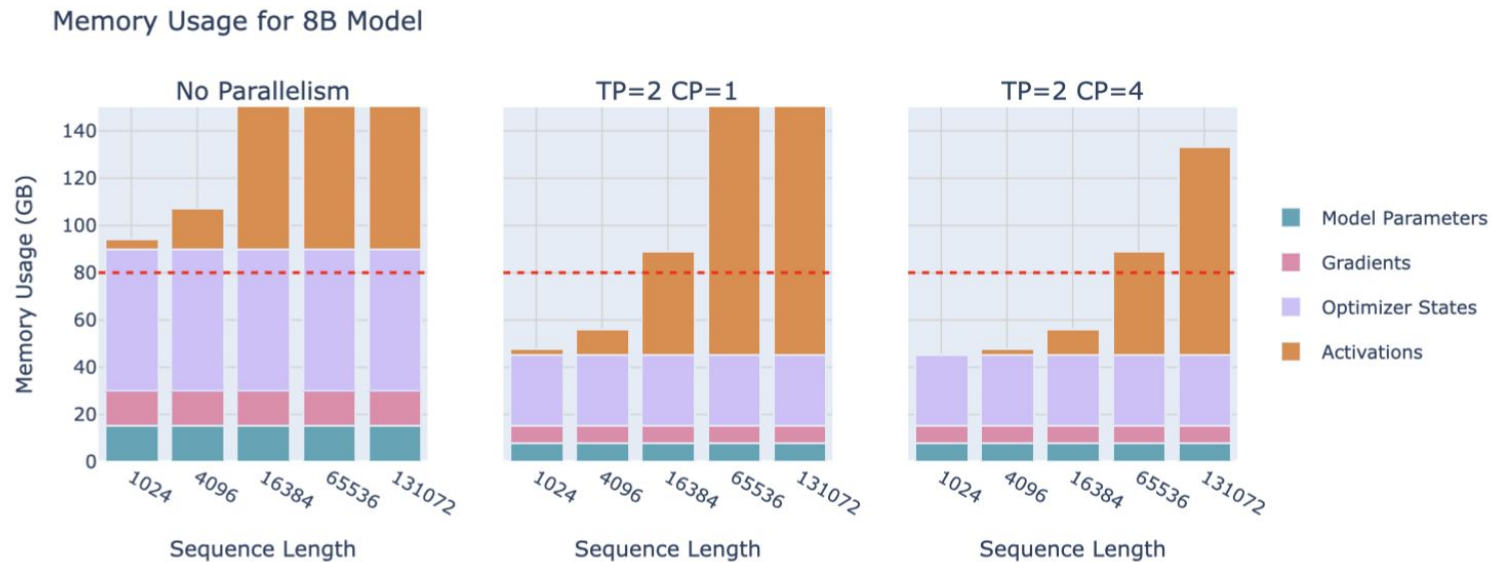
Tensor parallelism



Sequence parallelism

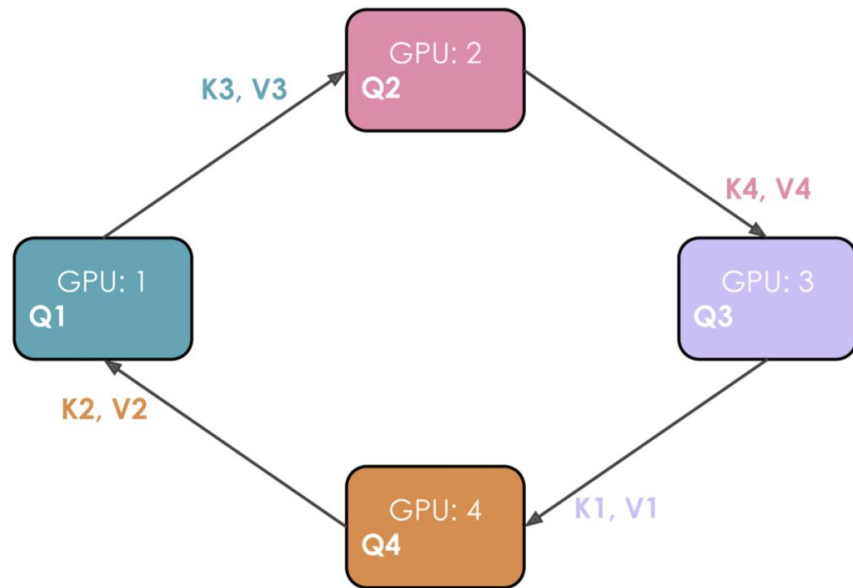


Context parallelism



Context parallelism

Ring attention:



Method	Memory savings apply specifically on...	Parallel/sharding dimension	Disadvantage
DP	Activations (reduce local batch size)	Batch	Limited by max batch size
PP	Model parameters	Model layers	Idle bubble and complex schedules
TP+SP	Model parameters and activations	Hidden dimension/sequence length	Requires high- bandwidth communication
CP	Activations	Sequence length	Adds communication overhead in attention modules
EP	Experts parameters	Experts dimension	Requires MoE layers, adds routing communication overhead
ZeRO-1	Optimizer states	Sharded among DP replicas	Params communication overhead
ZeRO-2	Optimizer states and gradients	Sharded among DP replicas	Params communication overhead
ZeRO-3	Optimizer states, gradients, and model parameters	Sharded among DP replicas	Params communication overhead

1. GPU-rich case 🥰 - when you have plenty of GPUs available:

- For models under 10B parameters, you can use a single parallelism technique, e.g. tensor parallelism or ZeRO-3/DP with full recompute across 8 GPUs.
- For models between 10B-100B parameters requiring more than 8 GPUs, you have several options:
 - Combining tensor parallelism (TP=8) with pipeline parallelism
 - Combining tensor parallelism (TP=8) with data parallelism (ZeRO-3)
 - Using only ZeRO-3 (i.e., pure data parallelism)
- At 512+ GPU scale, pure data parallelism/ZeRO-3 will start to become inefficient due to communication cost - it can be better to then combine DP with either tensor or pipeline parallelism.
- At 1024+ GPU scale, a recommended setup may be tensor parallelism (TP=8) with data parallelism (ZeRO-2) and pipeline parallelism.

We focus on fitting a single instance for now - even though we may use DP for ZeRO to achieve this goal - we're only interested here in the model parameters memory savings that it provides when used with ZeRO-3.

Special considerations:

- For very long sequences, you will probably want to add context parallelism across nodes.
- For Mixture of Experts architectures, it will be advantageous to use expert parallelism across nodes.

2. GPU-poor case 😞 - when you might be low on GPU resources:

- You can enable full activation recomputation to trade some compute for memory (and train a bit more slowly).
- You can increase gradient accumulation to process larger batches with limited memory.